

Using BDD and SBVR to refine business goals into an Event-B model: a research idea

Fabio Levy Siqueira^{*‡}, Thiago C. de Sousa[†], Paulo S. Muniz Silva^{*‡}

^{*}Escola Politécnica da Universidade de São Paulo, Brazil

Email: {levy.siqueira, paulo.muniz}@usp.br

[‡]IPT - Instituto de Pesquisas Tecnológicas, Brazil

[†]Centro de Tecnologia e Urbanismo da Universidade Estadual do Piauí, Brazil

Email: thiago@uespi.br

Abstract—The transition from a requirements document to a formal specification in Event-B is usually manual and ad-hoc. In order to bridge this gap, we propose a method based on Behavior-Driven Development, an agile approach, and that uses a structured natural language conformant to the formalism of the Semantics of Business Vocabulary and Business Rules (SBVR) standard. This method will successively refine a list of high-level business goals into an Event-B model using transformations. In this paper we present our research idea, describing the steps of this method and showing an example based on the Train System scenario described by Abrial.

Keywords—formal methods, requirements, method, Event-B, BDD, SBVR

I. INTRODUCTION

The transition from the requirements phase to the formal specification phase is one of the most painful steps and is still an open issue. Abrial [1] proposes a format for the requirements description, but he does not describe a method and the transformation into an initial formal specification is manual. Other works propose automatic transformations from a requirements model, such as Use Case [2] or KAOS [3], but the initial refinements depend on the original method used, and there is no traceability from these refinements to the Event-B model. Works such as [4] and [5] combine requirements refinement and formal methods, but the involvement of stakeholders in the refinement process is limited.

In this work we present our research idea of using Behavior-Driven Development (BDD) [6], [7], an agile approach, as the starting point for modeling in Event-B. More specifically, we propose the combination of a BDD template, to represent features, user stories, and scenarios, with a structured English language defined via the formalism provided by SBVR [8], in order to obtain a refinement from requirements into an Event-B model. Moreover, at the end of each refinement step, it is also possible to make a formal verification using the Event-B proof obligation mechanism, which contributes to reduce model inconsistencies in the early phases of the development process. To describe this idea, we present a method (steps and roles) and an example of its use. The example is inspired on the Train System scenario described by Abrial [1]. Our goal is to create a tool that automates the transformation of requirements models into Event-B models.

In the next section we provide some background about Event-B, BDD and SBVR. In the third section, we present the proposed method as well as an example of its use. Finally, we present the related work and finish this article with some discussions and ongoing work.

II. BACKGROUND

A. Event-B

Event-B [1] is a state model-based formal method for modeling systems based on predicate logic and set theory where the refinement mechanism and the consistency checking are guaranteed by mathematical proof obligations. The refinement mechanism allows us to build a model gradually by making it more and more precise (that is, from an abstract model to a concrete one); and the consistency checking permits us to verify the validity of the properties of a system. These features have been supported by an open-source platform (based on Eclipse IDE) named Rodin [9], that is constantly improved and extended by the community via plugins.

Since Event-B is based on sound mathematical foundation, we can reason about the model ensuring that the refinement steps can be demonstrated and that the properties of a system can be also proved to be consistent. For the verification of a refinement we have to check two situations: the guard of the refined event is stronger than the guard of the abstract event; and for each possible execution of concrete event, there is a corresponding of the abstract event such that the *gluing invariant* (a relation between abstract and concrete variables) is valid after the execution of both. For the verification of consistency checking we need to prove: *the invariant preservation*, that means that, before the execution of an event (including the initialisation event), the invariant is valid and should remain so after its application; and *the event feasibility*, that means that an event may always occur when the guard is true, ensuring the existence of a post-state.

The Event-B language has two components: contexts and machines. Machines and contexts may have several relationships: a machine refines another and a context extends another one. Furthermore, a machine sees one or more contexts. A context is used for static specification, including global sets, constants and axioms and its structure is “*context C1 extends C2 sets S constants C axioms A end*”, where *C1* and *C2* are

context names, S and C are list of set and constant, respectively, and A represents axiom rules. A machine is applied to dynamic specification, containing variables, invariants and events and is described usually by “*machine $M1$ refines $M2$ sees $C1$ variables V invariants I events E end*”, where $M1$ and $M2$ are machine names, $C1$ is a context and V , I and E are list of state variables, invariants and events respectively.

An event is composed by two elements: guards and its parameters, which are valid conditions for the occurrence of the event; and actions, which denote how the variables change. In general, an event has the form “*any x where G then A end*”, where x is a list of parameters, G is a list of guards and A is a list of actions.

B. BDD

Behavior-Driven Development (BDD) is an agile software development approach with an emphasis on requirements analysis and testing. It was created by North [6] to help applying test-driven development, but it then evolved to an approach that groups several techniques and methods [7].

According to Smart [7], the process of BDD starts by determining a set of high-level business goals. Based on these goals, the analyst should describe the features that the system must deliver. Features are then broken into user stories. According to Cohn, a user story “describes functionality that will be valuable to either a user or purchaser of a system or software” [10, p.4]. As an agile technique, the written description of a user story is not a specification, but a reminder for a conversation between development team and stakeholders [10]. In the next step of the process, the user stories are illustrated with examples, called scenarios. Scenarios represent the acceptance criteria of the user story. Finally, the scenarios are translated to automated tests, usually with support of frameworks such as JBehave [11] or Cucumber [12].

There are different templates for representing features and stories in BDD [7]. Scenarios, on the other hand, are usually represented using the template suggested by North [6]:

Given C
When E
Then O

where C describes the conditions in the context that must be true for the scenario be executed; E describes the event to be tested and that triggers the execution of the scenario [7]; and O represents the expected outcome or state of the system.

C. SBVR

Semantics of Business Vocabulary and Business Rules (SBVR) [8] is an OMG standard that allows documenting the semantic of business vocabularies and business rules. It can be used to create a rulebook, which is composed by a terminological dictionary or business vocabulary (terms and other representations, including definitional rules) and elements of guidance (including behavioral business rules). Among its characteristics, SBVR allows an unambiguous definition of business concepts and rules, which can be understood by business people.

Even though SBVR does not impose a language, the specification includes the description of SBVR Structured English [8]. The statements in this language use four font styles to represent the following concepts:

- **Term**: representing noun concepts.
- **Name**: representing individual concepts such as proper nouns and numbers.
- **Verb**: representing a verb and/or a preposition. Verbs are normally used in singular and active forms.
- **Keyword**: words that can be combined, such as “the”, “an”, “what”, and phrases for logical formulations.

From a logical perspective, SBVR allows expressing quantification (e.g. “each”, “some”, “more than one”), logical operations (e.g. “and”, “or”, “neither”), and modal operations (e.g. “it is necessary that”, “it is prohibited that”).

III. BDD2EVENT-B

Our goal is to have a sequence of refinement steps, supported by automatic transformations, to obtain a complete model of the system in Event-B from high-level business goals. In addition to smooth the transition from requirements to specifications, the proposed method also allows making formal verification using the power of Event-B proof obligation mechanism in each refinement step, which contributes to minimize model inconsistencies in the initial phases of the development process.

As these steps involve requirements refinement, it is important that the domain specialist is able to understand the requirements and help the requirements analyst to specify them. To achieve this goal, we propose using a method based on BDD. Similarly to the method proposed by Abrial [1], BDD suggests applying a sequence of refinements. Yet, the stakeholders should participate actively in the process: they write the requirements model with the help of a developer. Another reason for using BDD is the similarity of the template to write scenarios and the Event-B model, what makes the transformation easier.

Even though BDD uses natural language, we decided to use SBVR Structured English for the following reasons. First, it is a controlled natural language [13], and, therefore, can be processed by tools. Second, it allows using quantification, logical operators, and modal operations, what is important for a specification. Finally, it was designed to be used and understood by business specialists.

In the proposed method, the customer, a domain specialist and a requirements analyst (with background in formal methods) work together to create a requirements model written in SBVR structured English. In addition to goals, features, user stories and scenarios, the requirements model must also contain a SBVR vocabulary, which defines the terms used and the relationship between them. This model can be seen as a SBVR rulebook.

The execution of the method is exemplified in a system inspired on the Train System example described by Abrial [1]. This system should help a train agent to control trains crossing a track network. In this domain, the track network is made of

blocks, and contains points (a device that splits a track in two directions) and crossings (a device that makes two tracks cross each other). The paths trains may follow in the network are called routes, and they are defined as a sequence of blocks. A signal is located before the first block of the route, protecting it. If the signal is red, the train cannot use the corresponding route; if it is green, the train can move safely in the route.

The method is composed of four steps, based on the BDD method described by Smart [7]: (1) determining the high-level business goals, (2) determining the software features, (3) refine the features into user stories, and (3) define the acceptance criteria of the user stories. For each step we describe the roles involved, input and outputs, and how a tool could be used to automatically transform models.

A. Step 1: Determining the high-level business goals

The customer and a requirements analyst establish the goals of the project and the high-level business goals of the application. As there is no input for this step, there is no refinement involved. The output is a list of business goals described in unrestricted natural language.

In the Train System example, there is only one goal:

The goal of the train system is to safely control trains moving on a track network.

B. Step 2: Determining the software features

The customer, domain specialists, and a requirements analyst describe the system features. The list of goals created in the previous step is manually refined into a list of features. Each feature should have a name and its description should use the format proposed by Smart [7]:

In order to G

As a S

I want to D

where G represents a business goal or business value, S represents a stakeholder, and D is a description of the feature. The text is written in SBVR Structured English and an initial rulebook is created, defining the important terms in the text. This step may also define high-level feature-wide nonfunctional requirements, such as safety, security, etc.

Another output from this step is an Event-B model, what could be obtained from an automatic transformation from the SBVR terms in the list of features. This model describes the static part of the system, which is extracted from the concepts presented in the rulebook, as well as its dynamic part, which is derived from the feature text.

In the Train System example, one of the features obtained from the refinement of the goal is presented in Figure 1, with its vocabulary and the related Event-B code. Simplistically, the implicitly-understood concepts (“Train Agent”, “Block”, etc.) generate sets in the Event-B context model. Since the “in order to” clause means the realization of a benefit, it produces an event in the Event-B machine model (“a train cross a track network” generates an event named “aTraincrossTNetw”). As the “I want to” clause stands for doing something in the BDD

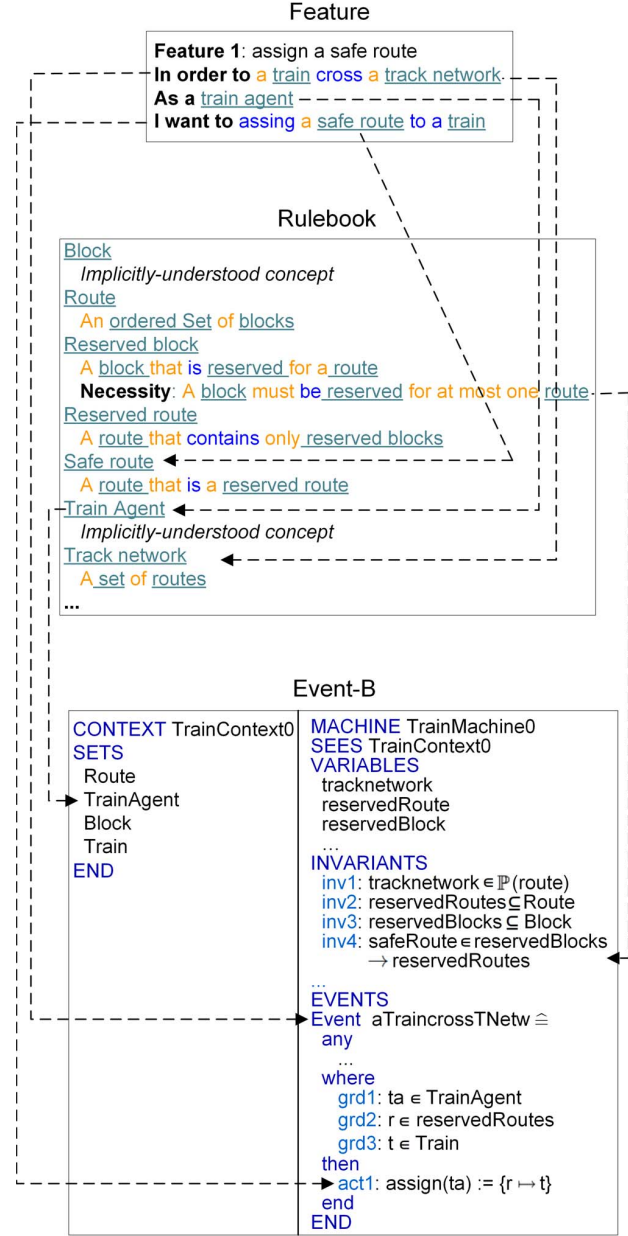


Fig. 1. A feature and a fragment of the rulebook, and the Event-B model created (transformations are represented as arrows).

view, it is translated to an action of that event: the “assign” verb maps a “reserved route” to a “Train”, noting that we simplistically define this kind of route as a “safe route”.

For the sake of simplicity, among the safety conditions [1], we illustrate just one safety condition for a “safe route”. This safety condition is expressed in SBVR as an alethic structural rule of necessity, meaning that it cannot be violated. Necessity rules are then mapped to the clause INVARIANT of an abstract machine. The rulebook should have entries that help the definition of a “safe route”. Other invariants are

derived from the rulebook, such as that a “tracknetwork” is a powerset of “route”. Moreover, the stakeholder (“train agent”), presented in the “As a” clause, becomes an event parameter (“ta”), an important element to the feasibility of the event.

C. Step 3: Refine the features into user stories

After the formal verification of the vocabulary consistent provided by the Event-B proof obligation mechanism using the Rodin platform [9], a domain specialist and the requirement analyst break each feature into one or more user stories. Each story is a manual refinement of a feature, and should use the format proposed by North [6]:

As a S
I want F
So that B

where S represents the stakeholder that will benefit from the story, F represents a feature, and B represents the business value of the story. As in the features, the text should use the SBVR, evolving the rulebook with other terms.

The rulebook created could be transformed automatically into an Event-B model, which is a refinement of the previous Event-B model. Another output of this step is the traceability between the features and user stories.

In the example, the refinement of the feature created for the previous step is presented in Figure 2. A story “Check the status of routes” was created, and the rulebook was refined, adding other terms and adding definitions for others (for instance, a definition for Block and the limitation of 2 trains, according to the dialogues among the stakeholders). These models were also transformed into a refined Event-B specification. The “so that” clause produces a refined event of the event generated by the “in order to” clause of the feature 1 (“trains may follow a safe route” generates an event named “aTrainfollowaRoute”, which refines the previous “aTraincrossTNetw”).

The rulebook also extends the Event-B context, for example, refining the “Train” concept with an axiom stating that Train is a finite set, and refining invariants, such as “occupiedblock” which is a powerset of “Block”, related to the refinement of the “Block” concept in the rulebook. Once again, alethic necessity rules are transformed into invariants, and the user story “I want to” clause is mapped to an action of a refined event of the “aTrainfollowaRoute” event that reserves a “route” to a “Train Agent”. The “I want to” clause (“check the reservation status of routes”) is transformed into the action “isReserved”, which allows a train agent query the reservation status of routes for specific trains. For the sake of simplicity, we do not show this reservation of a route by a “Train Agent”, but the query of the reservation status of a route.

D. Step 4: Define the acceptance criteria of the user stories

In the last step, after the formal verification of the consistency between the features and the stories, a domain specialist and the requirements analyst use the user stories generated in Step 3 to manually create scenarios. A scenario represents an acceptance criteria of the user story and is written using the

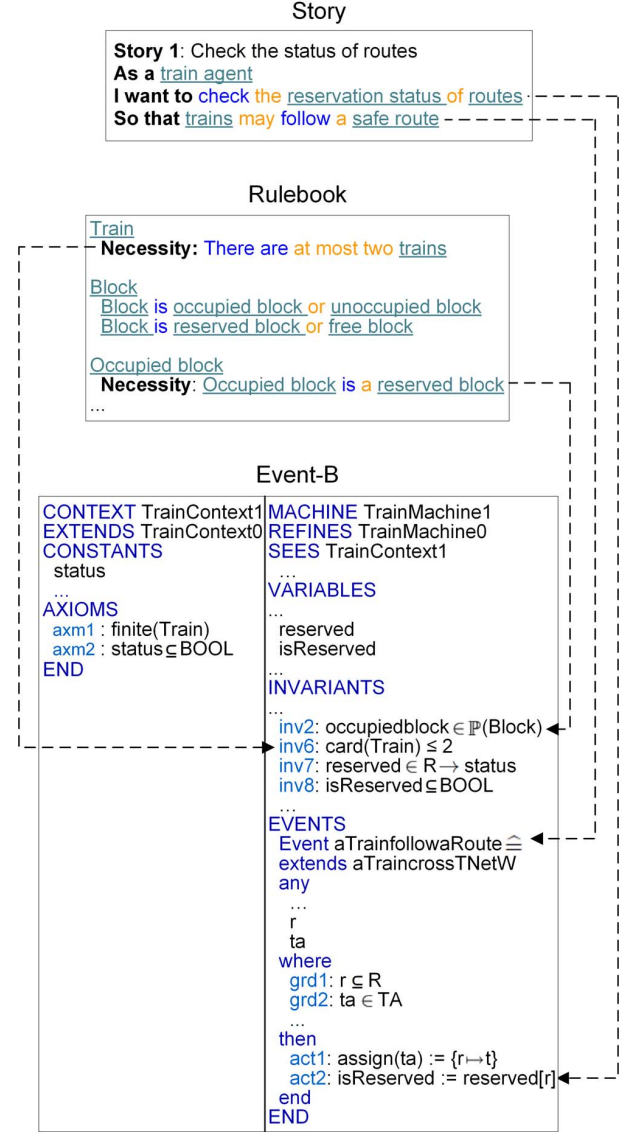


Fig. 2. A story and a fragment of the rulebook, and the Event-B model created (transformations are represented as arrows).

given-when-then template [6], [7]. Once again, the rulebook should be improved.

The scenarios created in this step should be more abstract than the ones suggested by BDD frameworks. Instead of specifying input and output values, what would be used by tools to generate test cases, the scenarios should represent conditions and results - as North originally suggested when proposing BDD [6]. For example, instead of describing that the train is in “block A”, the scenario should describe that the train is in a block with a red signal (which “block A” would be an instance). This allows the requirements analyst to concentrate on the logic of the scenario instead of concrete details. Moreover, it allows transforming a scenario into an Event-B model, while a concrete scenario would only give a

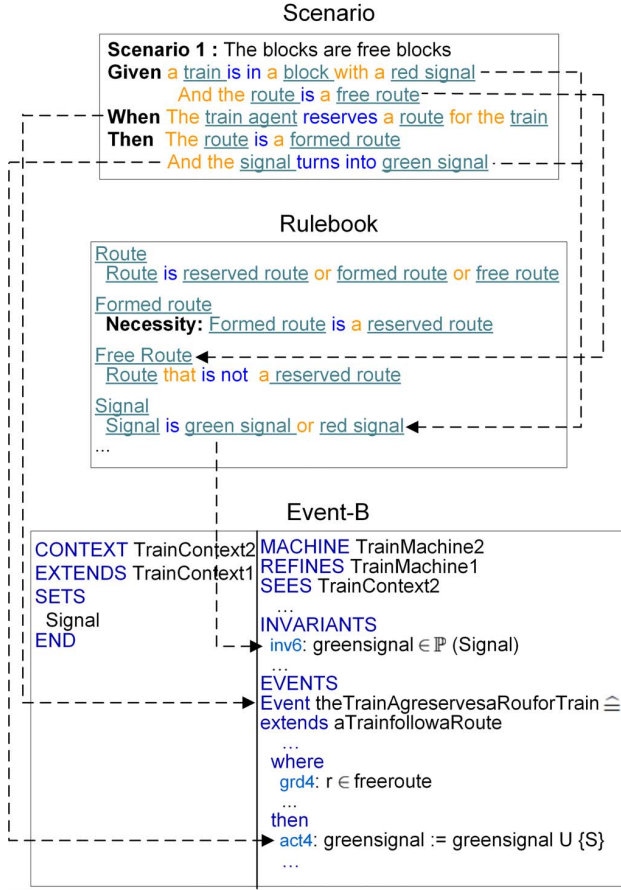


Fig. 3. A scenario and a fragment of the rulebook, and the Event-B model created (transformations are represented as arrows).

test case for the Event-B model.

The scenarios and rulebook could be transformed automatically into an Event-B model. The similarity of the scenario format and an Event-B machine favours the simplification of this transformation. Another output of this step is the traceability between the scenarios and story - as each scenario is related to exactly one story.

In Figure 3, it is presented the main scenario in which the blocks of a route are free, making possible to reserve the route. For simplicity, we do not take into account safety conditions and also assume that user stories related to the reservation of routes were produced in Step 3 and transformed into events of the abstract machine refinement generated in this step. The vocabulary is refined by defining other terms, such as a signal. The scenario and vocabulary are transformed into a second refined Event-B model. The *given* clause generates some event parameters (“freeroute”), the *when* clause produces a refined event (“theTrainAgreservesaRouforTrain”) of the event generated by a user story that reserves a route to a train, and the *then* clause is mapped to actions in the refined event, i.e. “Signal” becomes a “greensignal”. The improvement of the rulebook is used to extend the Event-B context and to

refine/add invariants (“greensignal” is a powerset of “Signal”) in the Event-B machine.

After the verification of the Event-B model generated by the last step, i.e. the consistency between stories and scenarios, the transition from the requirements phase to the formal specification phase must be complete. So, the requirements analyst should work together a formal methods specialist in order to continue the typical Event-B development process, which can include the addition of new invariants and properties, the checking of new proof obligations and the detailing of the model by new refinements until the point where it is ready for the generation of the source code, usually in C or Ada language.

IV. RELATED WORK

Some works propose requirements refinement approaches that result in models that allow limited formal reasoning. For example, there are the goal-oriented requirements engineering approaches KAOS [14] and Tropos [15], and the ontology-based framework proposed by Li et al. [16].

Other works propose a transformation from an existing requirement model into an Event-B model. Ponsard and Devroey [3] describes a transformation from a KAOS model to an Event-B model. Recently, [2] has described how to translate Use Cases Models into an Event-B model. However, our approach combines stepwise refinements and transformations into the formal model. For each refinement in the requirements model there is a refinement in the Event-B model, what improves traceability and allows a continuous verification of the model.

There are also some approaches that combine requirements refinement and formal methods. In [5] is proposed the combined use of Jackson’s WRSPM approach with Event-B in order to provide the foundation for tracing requirements to formal refinements. In [4] is suggested the use of the UML-B and Event Refinement Structures (ERS) approaches in order to assist the mapping from requirements to an Event-B model and the subsequent organization of refinement levels. However, our approach propose the use of the SBVR, which allows the involvement of stakeholders in the refinement process.

V. DISCUSSIONS AND ONGOING WORK

This paper presents an ongoing work to create a method to successively refine high-level business goals into an Event-B model, using transformations. The method is based on BDD and uses SBVR as controlled natural language. We exemplified the proposed steps using simple fragments of the Train System example described by Abrial [1].

The main benefit of this proposal is to bridge the gap between a formal specification and high-level business goals. The use of SBVR, a controlled language created to help the business community, allows a direct involvement of domain specialists in this process. Formal models in Event-B are obtained by transformations of requirements artifacts, with the support of SBVR formalism. On the other hand, the use of a method based on BDD allows successive refinements of the

requirements model, from goals to scenarios. Another benefit is to allow intermediate verification in requirements artifacts. A requirements analyst can use the Event-B model generated by each step, for instance, to find conflicts in stakeholders definitions and scenarios. In this way it is possible to visualize at least three types of formal verification during the development process: a) the vocabulary consistency, since it was built along the dialogues and during the process of learning the application domain; b) the consistency between the features and the stories, based on the definitions provided by the features (stories introduce more elements in the vocabulary) and, evidently, with the vocabulary already made consistent; c) the consistency between stories and scenarios, based on the stories (for the similar reason) with the vocabulary already made consistent.

There are several ongoing work to obtain a complete method. As our goal is to create a tool, we need to provide a mapping from BDD models into Event-B models in order to obtain an automatic transformation. This involves a further analysis on SBVR and a mapping from its formal model to Event-B constructs. We also need to improve the method, providing guidance for each step. Another future work is to investigate which proof obligations can be done in the intermediate models in order to make the requirements analyst task easier.

REFERENCES

- [1] J.-R. Abrial, *Modeling in Event-B: System and Software Engineering*, 1st ed. New York, NY, USA: Cambridge University Press, 2010.
- [2] R. Murali, A. Ireland, and G. Grov, "UC-B: Use Case Modelling with Event-B," in *Abstract State Machines, Alloy, B, TLA, VDM, and Z: 5th International Conference, ABZ 2016, Linz, Austria, May 23-27, 2016, Proceedings*, M. Butler, K.-D. Schewe, A. Mashkoor, and M. Biro, Eds. Springer International Publishing, 2016.
- [3] C. Ponsard and X. Devroey, "Generating high-level Event-B system models from KAOS requirements models," *Actes du XXIXème Congrès INFORSID*, pp. 317–332, 2011.
- [4] E. Alkhamash, M. Butler, A. S. Fathabadi, and C. Cirstea, "Building traceable Event-B models from requirements," *Science of Computer Programming*, vol. 111, Part 2, pp. 318 – 338, 2015, special Issue on Automated Verification of Critical Systems (AVoCS 2013).
- [5] M. Jastram, S. Hallerstede, M. Leuschel, and A. G. Russo, Jr., "An approach of requirements tracing in formal refinement," in *Proceedings of the Third International Conference on Verified Software: Theories, Tools, Experiments*, ser. VSTTE'10. Berlin, Heidelberg: Springer-Verlag, 2010, pp. 97–111.
- [6] D. North, "Introducing BDD," *Better Software Magazine*, Mar. 2006.
- [7] J. F. Smart, *BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle*. Manning Publications Company, 2014.
- [8] OMG, "Semantics of business vocabulary and business rules," 2015, version 1.3.
- [9] J.-R. Abrial, M. Butler, S. Hallerstede, T. S. Hoang, F. Mehta, and L. Voisin, "Rodin: an open toolset for modelling and reasoning in event-b," *International Journal on Software Tools for Technology Transfer*, vol. 12, no. 6, pp. 447–466, 2010. [Online]. Available: <http://dx.doi.org/10.1007/s10009-010-0145-y>
- [10] M. Cohn, *User Stories Applied For Agile Software Development*. Addison-Wesley, 2004.
- [11] T. H. C. G. North, Dan, Keogh, Elizabeth, "Jbehave," 2017, [Online; accessed 17-January-2017]. [Online]. Available: <http://www.jbehave.org/>
- [12] Cucumber Limited, "Cucumber," 2017, [Online; accessed 17-January-2017]. [Online]. Available: <https://cucumber.io/>
- [13] T. Kuhn, "A survey and classification of controlled natural languages," *Computational Linguistics*, vol. 40, no. 1, pp. 121–170, March 2014.
- [14] A. van Lamsweerde, *Requirements Engineering: From System Goals to UML Models to Software Specifications*, 1st ed. Chichester, England ; Hoboken, NJ: Wiley, Feb. 2009.
- [15] J. Castro, M. Kolp, and J. Mylopoulos, "A Requirements-Driven Development Methodology," in *Advanced Information Systems Engineering*, ser. Lecture Notes in Computer Science, K. R. Dittrich, A. Geppert, and M. C. Norrie, Eds. Springer Berlin Heidelberg, Jun. 2001, no. 2068, pp. 108–123.
- [16] F.-L. Li, J. Horkoff, A. Borgida, G. Guizzardi, L. Liu, and J. Mylopoulos, "From Stakeholder Requirements to Formal Specifications Through Refinement," in *Requirements Engineering: Foundation for Software Quality*, ser. LNCS, S. A. Fricker and K. Schneider, Eds. Springer, Mar. 2015, no. 9013, pp. 164–180.